

Week -02

Write a program to create an B-tree and perform Search, Insertion and Delete operations ?

```
#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>

#include <math.h>

// Define the structure of a B-Tree Node
typedef struct BTreeNode {

    int *keys;          // Array of keys

    int m;              // Order of the B-Tree (user-defined)

    int t;              // Minimum degree (ceil(m/2))

    struct BTreeNode **C; // Array of child pointers

    int n;              // Current number of keys

    bool leaf;          // True if node is a leaf

} BTreeNode;

// Function to create a new B-Tree node
BTreeNode* createNode(int m, bool leaf) {

    BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));

    node->m = m;

    node->t = (int)ceil((float)m / 2); // minimum degree

    node->leaf = leaf;

    node->keys = (int*)malloc((m - 1) * sizeof(int));

    node->C = (BTreeNode**)malloc(m * sizeof(BTreeNode*));

    node->n = 0;

    return node;

}

// Function to traverse the B-Tree
void traverse(BTreeNode* root) {

    if (root != NULL) {

        int i;
```

```

    for (i = 0; i < root->n; i++) {
        if (!root->leaf)
            traverse(root->C[i]);
        printf("%d ", root->keys[i]);
    }
    if (!root->leaf)
        traverse(root->C[i]);
}
}

```

// Function to search a key in the B-Tree

```

BTreeNode* search(BTreeNode* root, int k) {
    int i = 0;
    while (i < root->n && k > root->keys[i])
        i++;

    if (i < root->n && root->keys[i] == k)
        return root;

    if (root->leaf)
        return NULL;

    return search(root->C[i], k);
}

```

// Split the child of a node during insertion

```

void splitChild(BTreeNode* parent, int i, BTreeNode* y) {
    int t = y->t;
    BTreeNode* z = createNode(y->m, y->leaf);
    z->n = t - 1;

    // Move t-1 keys to new node z
    for (int j = 0; j < t - 1; j++)
        z->keys[j] = y->keys[j + t];
}

```

```

// Move children if not leaf
if (!y->leaf) {
    for (int j = 0; j < t; j++)
        z->C[j] = y->C[j + t];
}

y->n = t - 1;

for (int j = parent->n; j >= i + 1; j--)
    parent->C[j + 1] = parent->C[j];

parent->C[i + 1] = z;

for (int j = parent->n - 1; j >= i; j--)
    parent->keys[j + 1] = parent->keys[j];

parent->keys[i] = y->keys[t - 1];
parent->n++;
}

// Insert non-full node
void insertNonFull(BTreeNode* node, int k) {
    int i = node->n - 1;

    if (node->leaf) {
        while (i >= 0 && k < node->keys[i]) {
            node->keys[i + 1] = node->keys[i];
            i--;
        }
        node->keys[i + 1] = k;
        node->n++;
    } else {
        while (i >= 0 && k < node->keys[i])

```

```

    i--;

    if (node->C[i + 1]->n == node->m - 1) {
        splitChild(node, i + 1, node->C[i + 1]);
        if (k > node->keys[i + 1])
            i++;
    }
    insertNonFull(node->C[i + 1], k);
}
}

```

// Function to insert key into the B-Tree

```

void insert(BTreeNode** rootRef, int k) {
    BTreeNode* root = *rootRef;

    if (root->n == root->m - 1) {
        BTreeNode* s = createNode(root->m, false);
        s->C[0] = root;
        splitChild(s, 0, root);
        int i = (s->keys[0] < k) ? 1 : 0;
        insertNonFull(s->C[i], k);
        *rootRef = s;
    } else {
        insertNonFull(root, k);
    }
}

```

// Helper to find the predecessor

```

int getPredecessor(BTreeNode* node) {
    while (!node->leaf)
        node = node->C[node->n];
    return node->keys[node->n - 1];
}

```

```
// Helper to find the successor
```

```
int getSuccessor(BTreeNode* node) {  
    while (!node->leaf)  
        node = node->C[0];  
    return node->keys[0];  
}
```

```
// Merge two children of a node
```

```
void merge(BTreeNode* node, int idx) {  
    BTreeNode* child = node->C[idx];  
    BTreeNode* sibling = node->C[idx + 1];  
    int t = node->t;  
  
    child->keys[t - 1] = node->keys[idx];  
  
    for (int i = 0; i < sibling->n; i++)  
        child->keys[i + t] = sibling->keys[i];  
  
    if (!child->leaf) {  
        for (int i = 0; i <= sibling->n; i++)  
            child->C[i + t] = sibling->C[i];  
    }  
  
    for (int i = idx + 1; i < node->n; i++)  
        node->keys[i - 1] = node->keys[i];  
  
    for (int i = idx + 2; i <= node->n; i++)  
        node->C[i - 1] = node->C[i];  
  
    child->n += sibling->n + 1;  
    node->n--;  
  
    free(sibling->keys);  
    free(sibling->C);
```

```

    free(sibling);
}

// Fill the child if it has less than t-1 keys
void fill(BTreeNode* node, int idx) {
    int t = node->t;

    if (idx != 0 && node->C[idx - 1]->n >= t)
        borrowFromPrev(node, idx);
    else if (idx != node->n && node->C[idx + 1]->n >= t)
        borrowFromNext(node, idx);
    else {
        if (idx != node->n)
            merge(node, idx);
        else
            merge(node, idx - 1);
    }
}

```

```

// Borrow from previous sibling
void borrowFromPrev(BTreeNode* node, int idx) {
    BTreeNode* child = node->C[idx];
    BTreeNode* sibling = node->C[idx - 1];

    for (int i = child->n - 1; i >= 0; i--)
        child->keys[i + 1] = child->keys[i];

    if (!child->leaf) {
        for (int i = child->n; i >= 0; i--)
            child->C[i + 1] = child->C[i];
    }

    child->keys[0] = node->keys[idx - 1];
}

```

```

if (!child->leaf)
    child->C[0] = sibling->C[sibling->n];

node->keys[idx - 1] = sibling->keys[sibling->n - 1];

child->n += 1;
sibling->n -= 1;
}

```

// Borrow from next sibling

```

void borrowFromNext(BTreeNode* node, int idx) {

```

```

    BTreeNode* child = node->C[idx];

```

```

    BTreeNode* sibling = node->C[idx + 1];

```

```

    child->keys[child->n] = node->keys[idx];

```

```

    if (!child->leaf)

```

```

        child->C[child->n + 1] = sibling->C[0];

```

```

    node->keys[idx] = sibling->keys[0];

```

```

    for (int i = 1; i < sibling->n; i++)

```

```

        sibling->keys[i - 1] = sibling->keys[i];

```

```

    if (!sibling->leaf) {

```

```

        for (int i = 1; i <= sibling->n; i++)

```

```

            sibling->C[i - 1] = sibling->C[i];

```

```

    }

```

```

    child->n += 1;

```

```

    sibling->n -= 1;

```

```

}

```

// Function to remove key from node

```
void removeKey(BTreeNode* node, int k);
```

```
void removeFromLeaf(BTreeNode* node, int idx) {  
    for (int i = idx + 1; i < node->n; i++)  
        node->keys[i - 1] = node->keys[i];  
    node->n--;  
}
```

```
void removeFromNonLeaf(BTreeNode* node, int idx) {  
    int k = node->keys[idx];  
    if (node->C[idx]->n >= node->t) {  
        int pred = getPredecessor(node->C[idx]);  
        node->keys[idx] = pred;  
        removeKey(node->C[idx], pred);  
    } else if (node->C[idx + 1]->n >= node->t) {  
        int succ = getSuccessor(node->C[idx + 1]);  
        node->keys[idx] = succ;  
        removeKey(node->C[idx + 1], succ);  
    } else {  
        merge(node, idx);  
        removeKey(node->C[idx], k);  
    }  
}
```

```
void removeKey(BTreeNode* node, int k) {  
    int idx = 0;  
    while (idx < node->n && node->keys[idx] < k)  
        ++idx;  
  
    if (idx < node->n && node->keys[idx] == k) {  
        if (node->leaf)  
            removeFromLeaf(node, idx);  
        else  
            removeFromNonLeaf(node, idx);  
    }
```



```

} else {
    if (node->leaf) {
        printf("Key %d not found\n", k);
        return;
    }

    bool flag = (idx == node->n);
    if (node->C[idx]->n < node->t)
        fill(node, idx);

    if (flag && idx > node->n)
        removeKey(node->C[idx - 1], k);
    else
        removeKey(node->C[idx], k);
}
}

// Wrapper to delete key and adjust root
void deleteKey(BTreeNode** rootRef, int k) {
    BTreeNode* root = *rootRef;
    removeKey(root, k);

    if (root->n == 0) {
        BTreeNode* tmp = root;
        if (root->leaf)
            *rootRef = NULL;
        else
            *rootRef = root->C[0];
        free(tmp);
    }
}

```

```

// Main function to test
int main() {

```

```

int m = 4; // Order of the B-tree
BTreeNode* root = createNode(m, true);

int keys[] = {10, 20, 5, 6, 12, 30, 7, 17};
int n = sizeof(keys)/sizeof(keys[0]);

for (int i = 0; i < n; i++)
    insert(&root, keys[i]);

printf("Traversal after insertion: ");
traverse(root);
printf("\n");

deleteKey(&root, 6);
printf("Traversal after deleting 6: ");
traverse(root);
printf("\n");

deleteKey(&root, 13); // not found
printf("Traversal after deleting 13: ");
traverse(root);
printf("\n");

return 0;
}

```

Traversal of the constructed tree:

5 6 7 10 12 17 20 30

Traversal after deleting 6:

5 7 10 12 17 20 30